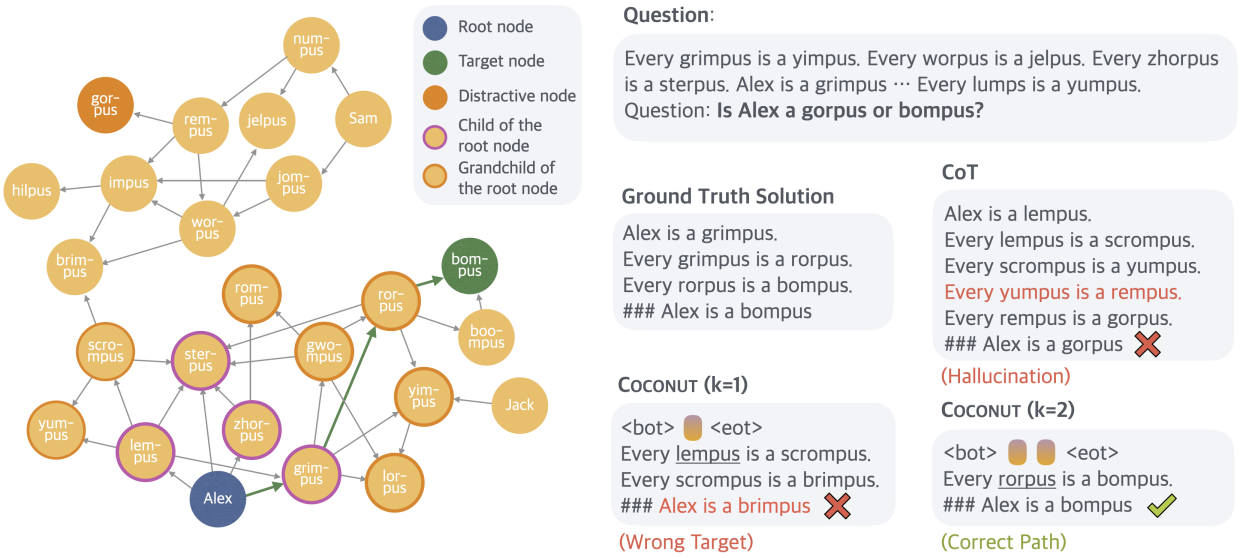

Figure 4 A case study of ProsQA. The model trained with *CoT* hallucinates an edge (*Every yumpus is a rempus*) after getting stuck in a dead end. Coconut (k=1) outputs a path that ends with an irrelevant node. Coconut (k=2) solves the problem correctly.

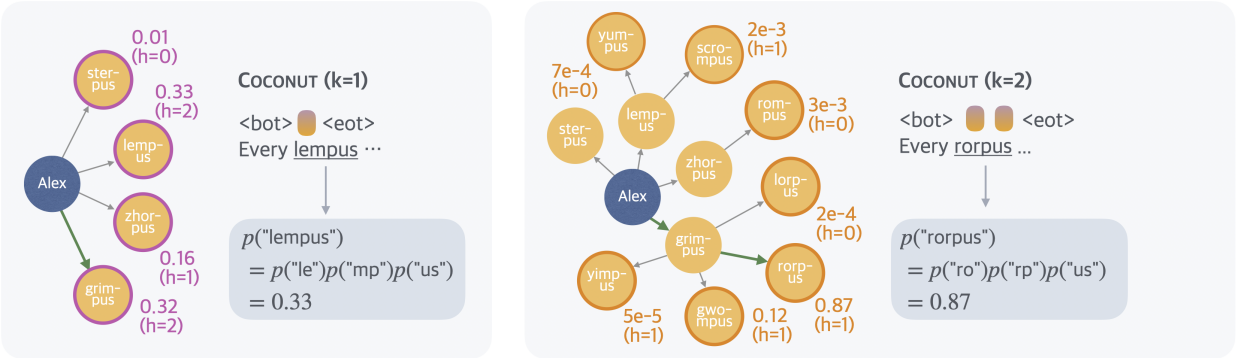

Figure 5 An illustration of the latent search trees. The example is the same test case as in Figure 4. The height of a node (denoted as h in the figure) is defined as the longest distance to any leaf nodes in the graph. We show the probability of the first concept predicted by the model following latent thoughts (e.g., "lempus" in the left figure). It is calculated as the multiplication of the probability of all tokens within the concept conditioned on previous context (omitted in the figure for brevity). This metric can be interpreted as an implicit value function estimated by the model, assessing the potential of each node leading to the correct answer.

We define the predicted probability of a concept following continuous thoughts as a value function (Figure 5), estimating each node's potential for reaching the correct target. Interestingly, the reasoning strategy employed by Coconut is not greedy search: while "lempus" initially has the highest value (0.33) at the first reasoning step (Figure 5, left), the model subsequently assigns the highest value (0.87) to "rorpus," a child of "grimpus," rather than following "lempus" (Figure 5, right). This characteristic resembles a breadth-first search (BFS) approach, contrasting sharply with the greedy decoding typical of traditional CoT methods. The inherent capability of continuous representations to encode multiple candidate paths enables the model to avoid making immediate deterministic decisions. Importantly, this tree search pattern is not limited to the illustrated example, but constitutes a fundamental mechanism underlying the consistent improvement observed with larger values of k in Coconut.

Figure 6 presents an analysis of the parallelism in the model's latent reasoning across the first and second thoughts. For the first thoughts (left panel), the cumulative values of the top-1, top-2, and top-3 candidate nodes are computed and plotted against their respective percentiles across the test set. The noticeable gaps between the three lines indicate that the model maintains significant diversity in its reasoning paths at this

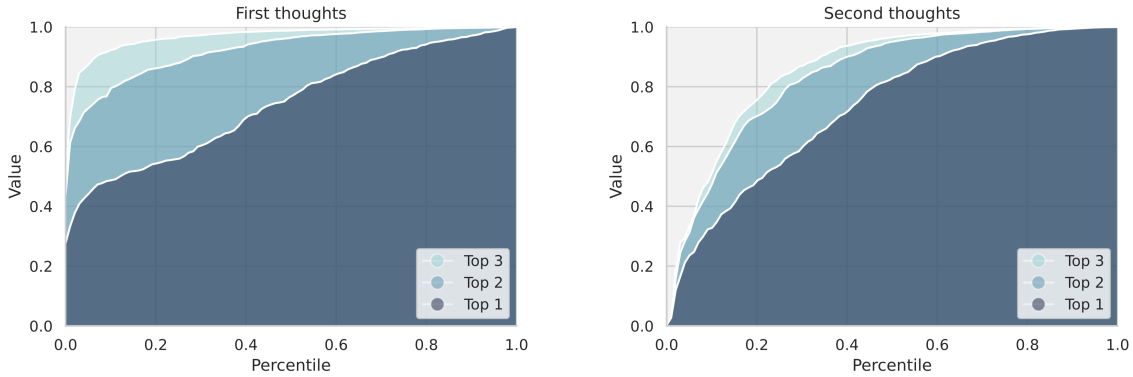

Figure 6 Analysis of parallelism in the first two steps of the latent tree search. The three curves in each panel depict the cumulative value of the top-1, top-2, and top-3 candidate nodes.

stage, suggesting a broad exploration of alternative possibilities. In contrast, the second thoughts (right panel) show a narrowing of these gaps. This trend suggests that the model transitions from parallel exploration to more focused reasoning in the second latent reasoning step, likely as it gains more certainty about the most promising paths.

4.4 Why is Latent Space Better for Planning?

Building upon the tree search perspective, we further examine why latent reasoning benefits planning tasks—specifically, why maintaining multiple candidate paths and postponing deterministic decisions enhances reasoning performance. Our hypothesis is that nodes explored in the early reasoning stages are inherently more challenging to evaluate accurately because they are farther from the final target nodes. In contrast, nodes positioned closer to potential targets, having fewer subsequent exploration possibilities, can be assessed accurately with higher confidence.

To systematically test this, we define the height of a node as its shortest distance to any leaf node and analyze the relationship between node height and the model's estimated value. Ideally, a correct node—one that can lead to the target node—should receive a high estimated value, whereas an incorrect node—one that cannot lead to the target node—should receive a low value. Empirical results across the test set (Figure 7) support our hypothesis: nodes with lower heights consistently receive more accurate and definitive probability evaluations. Conversely, nodes with greater heights exhibit more ambiguous evaluations, reflecting increased uncertainty.

These findings underscore the advantage of latent space reasoning. By delaying deterministic decisions and allowing exploration to proceed toward terminal states, latent reasoning significantly enhances the model's ability to differentiate correct paths from incorrect ones, thereby improving performance on complex, planning-intensive tasks compared to traditional greedy methods.

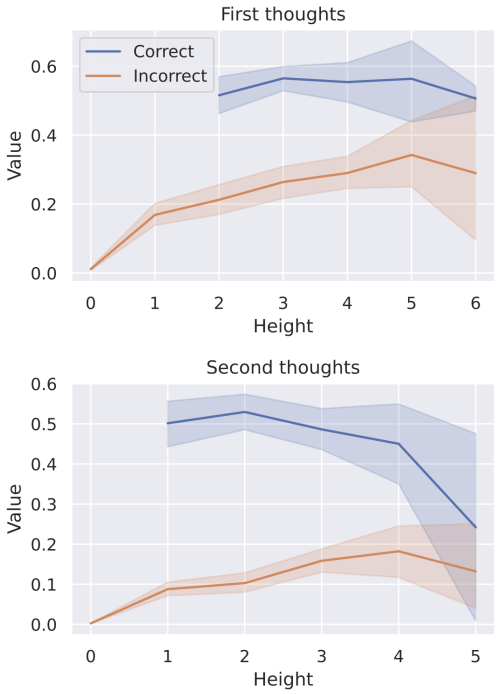

Figure 7 The correlation between the predicted value of correct/incorrect nodes and their heights.

5 Empirical Results with Coconut

After analyzing the promising parallel search pattern of Coconut, we validate the feasibility of LLM reasoning in a continuous latent space through more comprehensive experiments, highlighting its better reasoning efficiency over language space,

Method	GSM8k		ProntoQA		ProsQA	
	Acc. (%)	# Tokens	Acc. (%)	# Tokens	Acc. (%)	# Tokens
CoT	42.9 ±0.2	25.0	98.8 ±0.8	92.5	77.5 ±1.9	49.4
No-CoT	16.5 ±0.5	2.2	93.8 ±0.7	3.0	76.7 ±1.0	8.2
iCoT	30.0*	2.2	99.8 ±0.3	3.0	98.2 ±0.3	8.2
Pause Token	16.4 ±1.8	2.2	77.7 ±21.0	3.0	75.9 ±0.7	8.2
COCONUT (Ours)	34.1 ±1.5	8.2	99.8 ±0.2	9.0	97.0 ±0.3	14.2
- *w/o curriculum*	14.4 ±0.8	8.2	52.4 ±0.4	9.0	76.1 ±0.2	14.2
- *w/o thought*	21.6 ±0.5	2.3	99.9 ±0.1	3.0	95.5 ±1.1	8.2
- *pause as thought*	24.1 ±0.7	2.2	100.0 ±0.1	3.0	96.6 ±0.8	8.2

Table 1 Results on three datasets: GSM8k, ProntoQA and ProsQA. Higher accuracy indicates stronger reasoning ability, while generating fewer tokens indicates better efficiency. *The result is from Deng et al. (2024).

as well as its potential to enhance the model's expressivity with test-time scaling.

5.1 Experimental Setup

Math Reasoning. We use GSM8k (Cobbe et al., 2021) as the dataset for math reasoning. It consists of grade school-level math problems. To train the model, we use a synthetic dataset generated by Deng et al. (2023). We use two continuous thoughts for each reasoning step (i.e., $c = 2$). The model goes through 3 stages besides the initial stage. We then include an additional stage where still $3 \times c$ continuous thoughts are used as in the previous stage, but with all the remaining language reasoning chain removed. This handles the long-tail distribution of reasoning chains longer than 3 steps. We train the model for 6 epochs in the initial stage, and 3 epochs in each remaining stage.

Logical Reasoning. Logical reasoning involves the proper application of known conditions to prove or disprove a conclusion using logical rules. We use the ProntoQA (Saparov and He, 2022) dataset, and our newly proposed ProsQA dataset, which is more challenging due to more distracting branches. We use one continuous thought for every reasoning step (i.e., $c = 1$). The model goes through 6 training stages in addition to the initial stage, because the maximum number of reasoning steps is 6 in these two datasets. The model then fully reasons with continuous thoughts to solve the problems in the last stage. We train the model for 5 epochs per stage.

For all datasets, after the standard schedule, the model stays in the final training stage, until reaching 50 epochs. We select the checkpoint based on the accuracy on the validation set. For inference, we manually set the number of continuous thoughts to be consistent with their final training stage. We use greedy decoding for all experiments.

5.2 Baselines and Variants of Coconut

We consider the following baselines: (1) *CoT*, and (2) *No-CoT*, which were introduced in Section 4. (3) *iCoT* (Deng et al., 2024): The model is trained with language reasoning chains and follows a carefully designed schedule that "internalizes" CoT. As the training goes on, tokens at the beginning of the reasoning chain are gradually removed until only the answer remains. During inference, the model directly predicts the answer. (4) *Pause token* (Goyal et al., 2023): The model is trained using only the question and answer without a reasoning chain. However, different from *No-CoT*, special `<pause>` tokens are inserted between the question and answer, which provides the model with additional computational capacity to derive the answer. The number of `<pause>` tokens is set the same as continuous thoughts in COCONUT.

We also evaluate some variants of COCONUT: (1) *w/o curriculum*, which directly trains the model in the last stage. The model uses continuous thoughts to solve the whole problem. (2) *w/o thought*: We keep the multi-stage training, but don't add any continuous latent thoughts. While this is similar to *iCoT* in the high-level idea, the exact training schedule is set to be consistent with COCONUT, instead of *iCoT*, for a strict